

PROJECT INTRODUCTION

CodeBlue Nav

Hospital emergency routing that reroutes around blocked corridors and explains every decision.

TEAM

P .Vishal
B. Shashank
CH. Durgesh
D. Ganashyam

PROBLEM DOMAIN

Healthcare logistics · Graph
algorithms · Decision
support

PROBLEM STATEMENT

PS-5 · Hospital
Emergency Route
System

Minutes are lost inside the building

1.9M

neurons lost per minute in an untreated large-vessel stroke

≤ 60 min

AHA door-to-needle target for stroke

Real-world problem

- Teams navigate by memory and paper
- Blocked corridors go unseen
- Visitor apps ignore emergencies

Why it matters

- Stroke outcomes depend on minutes
- Detours eat the 60-minute window
- Mass casualty events multiply calls

Key challenges

- Instant rerouting on closures
- Defensible multi-call ranking
- Auditable AI output
- Must work offline

Routing that reacts, ranks, explains

CORE IDEA

Model the hospital as a weighted graph. Deterministic algorithms decide every route. AI only explains.

Key features

Dynamic rerouting: a blocked corridor recomputes every active route

Triage queue: calls ranked by severity, then ETA, re-ordered live

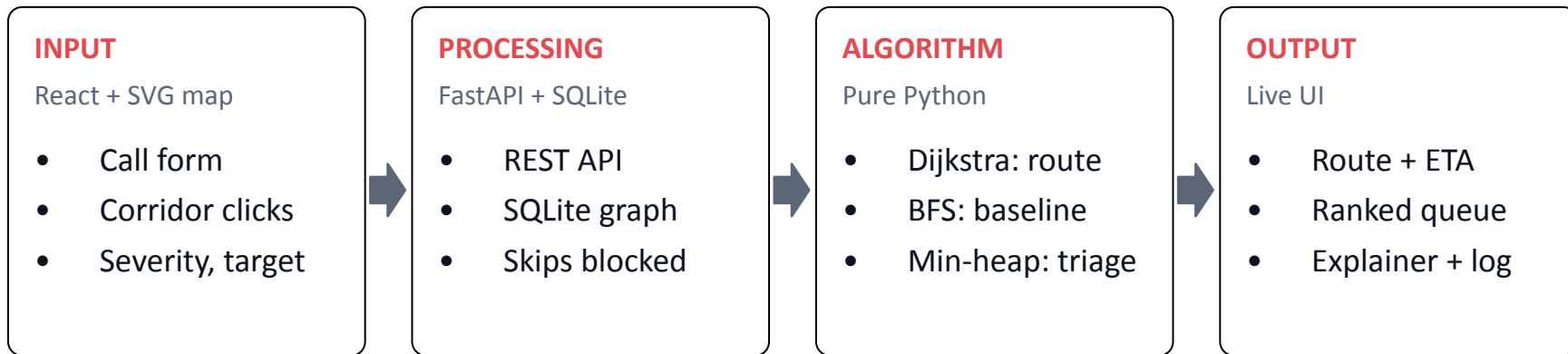
Explainer agent: one auditable sentence per decision, tool calls logged

Offline-first: local server, template narrator, replayable snapshot

Unique approach

- Algorithms decide, AI explains
- Dijkstra and BFS compared live
- Built for emergencies, not visitors
- Every decision auditable

Call in, explained route out



Agent layer: the LLM calls routing tools and writes one sentence. A template narrator covers any LLM failure.

USER JOURNEY

1 Trigger emergency

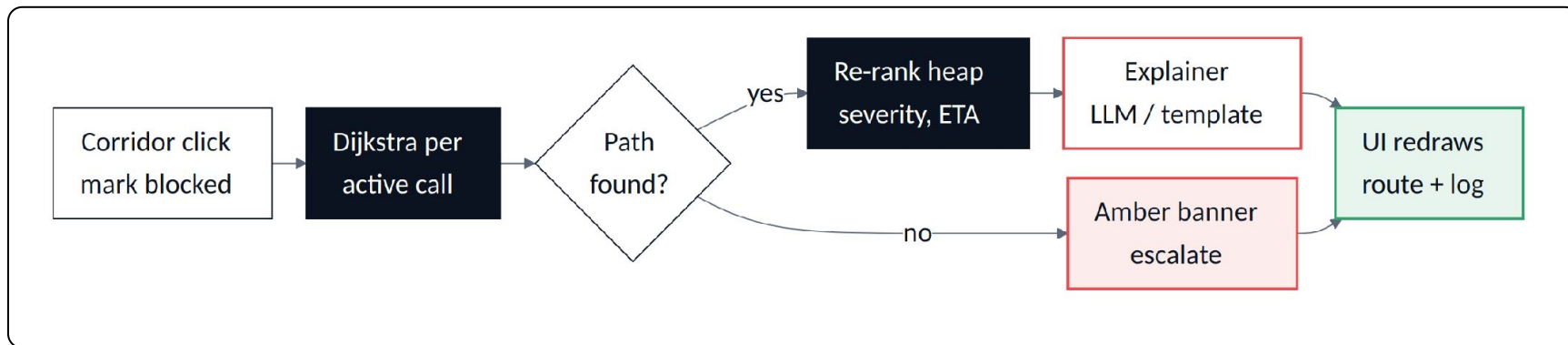
2 Route draws with
ETA

3 Block a corridor

4 Reroute with
reason

5 Queue re-ranks

What happens when a corridor closes



No viable path

Amber banner, call flagged for manual escalation, no crash

Shared destination

Second call gets a +30 s congestion penalty

LLM fails

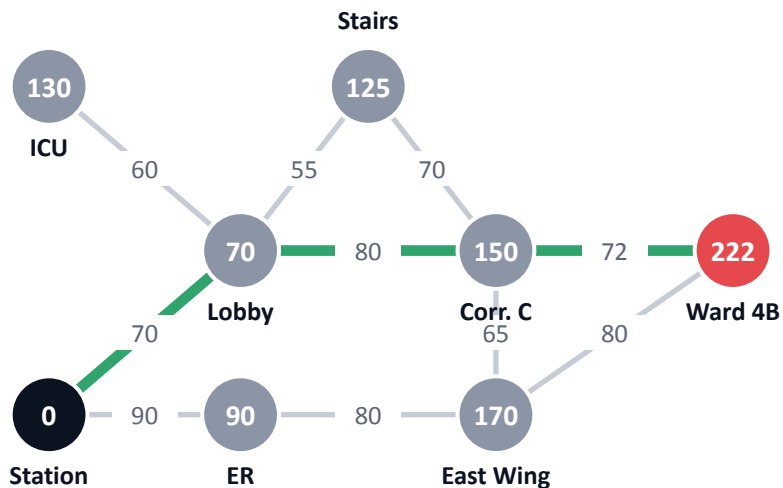
Over 4 s, error or 429: template narrator, tagged offline

Click spam

300 ms debounce, queue updates batched

Algorithms we can defend

Worked example: Dijkstra from Station to Ward 4B

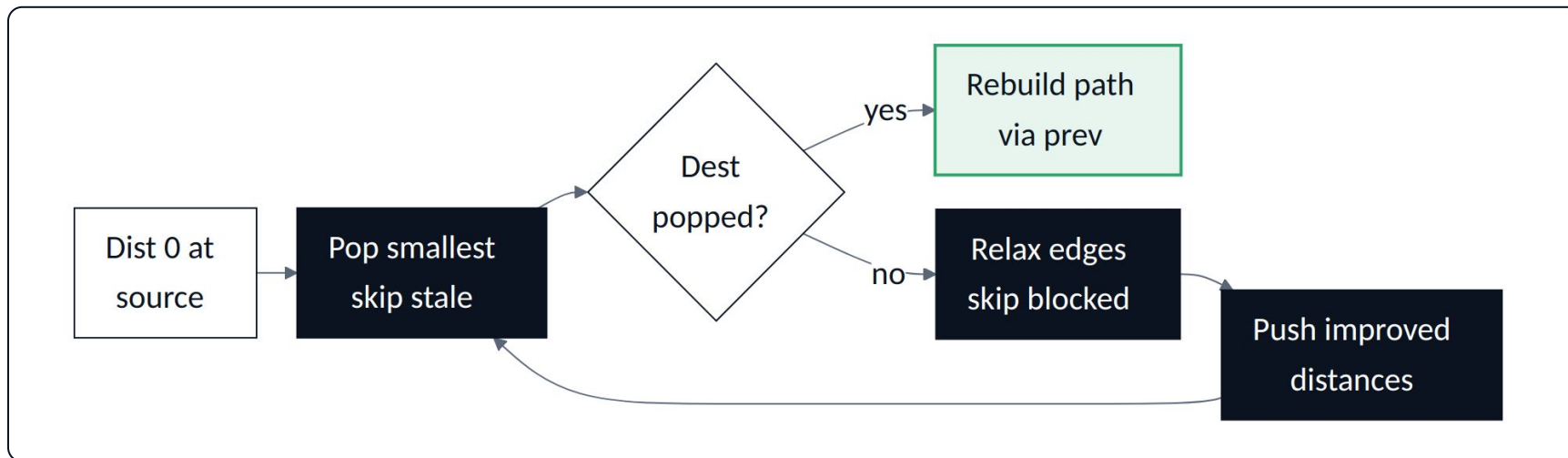


Algorithm	Why chosen
Dijkstra $O((V+E) \log V)$	Optimal for non-negative seconds
BFS $O(V+E)$	Fewest-corridor baseline
Min-heap $O(\log n)$ per call	Re-ranks calls as they arrive

Stack: React, SVG, FastAPI, SQLite, LLM tool calls. $V = 14$ nodes, $E = 22$ edges.

Settle order: S 0 → Lobby 70 → ER 90 → Stairs 125 → ICU 130 → Corr. C 150 → East Wing 170 → Ward 4B 222

How Dijkstra runs, step by step



Dijkstra: $O((V+E) \log V)$

Binary heap via `heapq`; stale entries skipped

BFS: $O(V+E)$

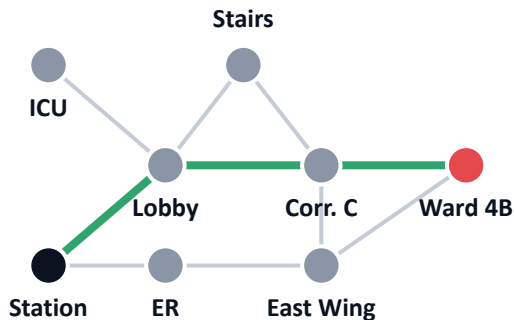
Hop-count baseline on the same graph

Triage heap: $O(\log n)$

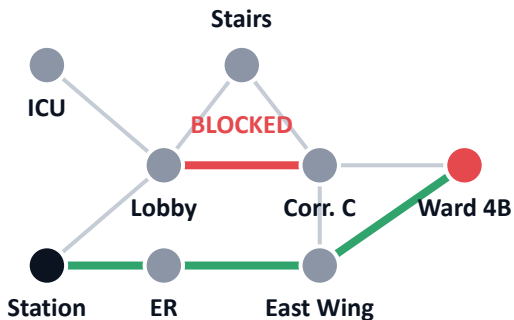
Severity high first, then lowest ETA

Block a corridor, watch it reroute

Before: 3m 42s via Corridor C



After: 4m 10s via ER and East Wing



```
[call] CODE BLUE Ward 4B
[tool] dijkstra -> 222 s
[event] Corr. C blocked
[tool] dijkstra -> 250 s
[explain] Detour via East Wing adds 28 s.
```

Practical benefits

- Faster dispatch decisions
- Fewer wrong turns
- Auditable, no patient data

Test cases: shortest path, forced detour, no-path error

3m 42s

before block

4m 10s

after block

+28 s

cost of the detour

From one floor to whole hospitals

NOW

Current achievement

- Live rerouting
- Triage + explainer
- Offline demo

NEXT

Future improvements

- Real floor plans
- Multi-floor routing
- Live staff reports

LATER

Scalability

- Larger graphs
- Multi-hospital rollout
- Incident dashboards

Takeaway: when a corridor closes, the system reroutes and says why.

Thank You

CodeBlue Nav